Eric Romero
3/23/2026
CSC 445

Homework 7

Algorithm Description:

The way I would handle this is by cutting it down rather than trying to inspect the entire grid like a maniac all at once. Start with the middle column of whatever part of the grid I am currently looking at, and then scan that whole column to find the highest location in it. That point is important because once I know it is the maximum in its column, I already know the cells above and below it cannot beat it. So, now I don't care about vertical neighbors anymore, at least not for that location. The only things that can still cause trouble are the cells directly to the left and right. From there, if neither the left or the right neighbor is higher, then done. It means the current location is greater than every adjacent location and is therefore a hill. If one of the horizontal neighbors is higher, then that shows which half of the grid is still worth chasing. If the left one is higher, I throw away the right half and keep searching on the left. If the right one is higher, I do the opposite. Then, repeat the process again on that smaller region: middle column, highest point in that column, check left and right, and either stop or keep cutting the search space in half. So the whole thing is basically controlled narrowing. I am not wandering through the grid cell by cell, and I am not trying to find the absolute highest point in the whole matrix either. I am just finding a strong point in the middle, seeing whether anything immediately to the side can overpower it, and using that to decide where to go next. That is why it stays efficient. Each step only scans one column, then throws away half the remaining columns, which gives a running time of $O(h \log w)$ for a grid with height h and width w.

Eric Romero
3/23/2026
CSC 445

Pseudocode

```
findHill(grid):
    Return dive (grid, 0, gridWidth - 1)

dive(grid, westEdge, eastEdge):
    Split = (westEdge + eastEdge) / 2

    // Scan middle column, find row that owns
    lockedRow = 0
    For row from 1 to gridHeight - 1:
        If grid.compare(row, split, locked, split) > 0:
            lockedRow = row
    driftLeft = false
    driftRight = false

    // Check grid, not current band
    If split > 0:
        If grid.compare(lockedRow, split - 1, lockedRow, split) > 0:
            driftLeft = true
    If split < gridWidth - 1:
        If grid.compare(lockedRow, split + 1, lockedRow, split) > 0:
            driftRight = true

    // No one to left or right beats this spot, so done
    If driftLeft == false and driftRight == false:
        Return new Location(lockedRow, split)

    // Otherwise, go to higher pressure point
    If drifttLeft:
        Return dive(grid, westEdge, split - 1)
    Else:
        Return dive(grid, split + 1, eastEdge)
```

Eric Romero
3/23/2026
CSC 445

Correctness:

The correctness of this algorithm comes from the way the dive keeps cutting into the grid and testing the strongest point in the middle of the current band. Each time dive(grid, westEdge, eastEdge) is called, it chooses the middle column split, then scans that column to find lockedRow, the row containing the highest location in that column. That chosen location, (lockedRow, split), is the pressure point the algorithm tests. Since it is the maximum entry in column split, it is automatically greater than the location directly above it, if one exists, and greater than the location directly below it, if one exists. In other words, once the algorithm locks onto the strongest point in the middle column, the vertical danger is already gone. The only remaining question is whether something immediately to the west or east is still stronger.

That is why the algorithm checks the left and right neighbors of (lockedRow, split). If neither neighbor is higher, then the chosen location is greater than every adjacent location that exists above, below, left, and right. That is exactly the definition of a hill, so returning new Location(lockedRow, split) is correct. If the left neighbor is higher, then the algorithm drifts left and recurses on the band from westEdge to split - 1. If the right neighbor is higher, it does the mirrored thing and recurses on the band from split + 1 to eastEdge. That move is safe because a higher adjacent neighbor proves that the terrain is still rising in that direction. Since all elevations are distinct and the grid is finite, repeatedly moving to a strictly higher adjacent location can never loop forever and cannot continue indefinitely. That rising path must eventually stop at some location that has no adjacent neighbor higher than it, and that stopping point is a hill. So whenever a dive chooses a direction, it is not guessing or wandering; it is following the only side that still contains an uphill trail to a real hill.

The recursion also has to terminate. Each recursive call throws away the middle column and one entire half of the remaining band, so the number of columns under consideration gets strictly smaller every time. Because the grid has finitely many columns, this process cannot continue forever. Eventually the dive reaches a call where the strongest point in the chosen middle column has no stronger neighbor to the left or right, and at that moment it returns a valid hill. Therefore, the algorithm always terminates, and when it terminates, it returns a location that satisfies the definition of a hill. That is why findHill(grid) is correct.

Eric Romero
3/23/2026
CSC 445

Complexity Analysis:

Time Complexity:

The algorithm is efficient because it does not search the whole grid blindly. In each call to dive, it focuses on one column only: the middle column of the current search band. It scans that column from top to bottom to find lockedRow, the row containing the strongest location in that column. Since there are h rows, that scan costs O(h) time. After that, the algorithm only does a constant amount of extra work by checking the left and right neighbors and deciding which direction to drift. So the real cost of one recursive call is O(h).

What makes the algorithm efficient is that after scanning the middle column, it throws away about half of the remaining columns and recurses only on the half that still contains the uphill direction. If the grid has width w, then the number of columns keeps getting cut down like this: w, then about w/2, then about w/4, and so on, until there is basically nothing left to cut. That gives the recurrence $T(w) = T(w/2) + O(h)$ because each level of recursion scans one column of height h, then continues on only half the columns.

Solving that recurrence gives a running time of $O(h \log w)$.

Space Complexity:

The algorithm does not build any extra arrays, tables, or copied versions of the grid. It just keeps track of a few variables like split, lockedRow, driftLeft, and driftRight, so each individual call uses only constant extra space. Because the algorithm is recursive, each call sits on the call stack until the deeper call finishes. Since the number of recursive calls is based on how many times the width can be cut in half, the recursion depth is O(log w). That means the total extra space used is O(log w) due to the call stack.